SESSION 2 – LAB PROGRAMS

Name: Arun.P **Reg. No:** 192372013

1. Remove Duplicate Elements and Find Space Complexity

Program (C):

```
#include <stdio.h>

int main(void) {
    int n, a[100], unique[100], count = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < count; j++) {
            if (a[i] == unique[j]) {
                found = 1;
                break;
            }
        }
        if (!found)
            unique[count++] = a[i];
    }

    printf("Unique elements: ");
    for (int i = 0; i < count; i++)
        printf("%d ", unique[i]);
    printf("\n");
    return 0;
}
```

Space complexity: O(n) auxiliary space for the new unique list (and O(n) for the input array in this implementation).
Time complexity: O(n^2) in the worst case due to checking each element against previously stored unique values.

2. Bubble Sort and Big-O Time Complexity

Program (C):

```
#include <stdio.h>

int main(void) {
    int n, a[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Time complexity: Worst and average case O(n^2). Best case O(n) when the array is already sorted and the swap flag stops early. **Auxiliary space:** O(1).

3. Binary Search in a Sorted Array

Program (C):

```
#include <stdio.h>

int main(void) {
    int n, arr[100], x;
    printf("Enter number of sorted elements: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter target: ");
    scanf("%d", &x);

    int low = 0, high = n - 1, found = 0;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) {
            found = 1;
            break;
        } else if (arr[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (found) printf("Element found\n");
    else printf("Element not found\n");
    return 0;
}
```

Requirement: The array must already be sorted in ascending order. **Time complexity:** O(log n), because the search range is halved each iteration. **Auxiliary space:** O(1) for this iterative version.

4. Sort in O(n log n) Without Built-in Sort

Merge sort is suitable: it guarantees O(n log n) time and uses O(n) extra space for the temporary array. This is typically the smallest practical auxiliary-space bound for a straightforward merge-sort implementation.

Program (C):

```
#include <stdio.h>
#include <stdlib.h>

void merge(int a[], int temp[], int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= right) temp[k++] = a[j++];

    for (i = left; i <= right; i++)
        a[i] = temp[i];
}

void mergeSort(int a[], int temp[], int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(a, temp, left, mid);
    mergeSort(a, temp, mid + 1, right);
    merge(a, temp, left, mid, right);
}

int main(void) {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n <= 0) return 0;

    int *a = malloc(n * sizeof(int));
    int *temp = malloc(n * sizeof(int));
    if (a == NULL || temp == NULL) {
        printf("Memory allocation failed\n");
```

```
        free(a);
        free(temp);
        return 1;
    }

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, temp, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");

    free(a);
    free(temp);
    return 0;
}
```

Time complexity: O(n log n) in best, average, and worst cases. **Auxiliary space:** O(n) for the temporary array, plus O(log n) recursion stack.